URBAN STAY

A INTERNSHIP PROJECT REPORT

Submitted by

Rathod Harsh

230143107017

In partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

Computer Engineering

Government Engineering College, Bharuch

Gujarat Technological University, Ahmedabad

July-2026

Government Engineering College, Bharuch

Old National Highway-8 Bholav, Bharuch, Gujarat 392002

CERTIFICATE

This is to certify that the internship report submitted along with the internship entitled "**Internship**" has been carried out by **Harsh Rathod (230143107017)** at **Logic Waves Tech** under my guidance in partial fulfilment of the requirements for the degree of **Bachelor of Engineering in Computer Engineering**, 8th Semester, Gujarat Technological University, Ahmedabad, during the academic year **2025–26.**

Prof. Shreyas Patel	Dr. Namrata Shroff
Internal Guide	Head of Department

COMPANY CERTIFICATE

LOGIC WAVES TECH

Issued on : 13-04-2026

ID No : LWT-ICC-11

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that Mr. Harsh Rathod, has successfully completed an internship with Logic Waves Tech from 12-01-2026 to 13-04-2026.

During this period, he worked in Full Stack *Development* and demonstrated sincerity, dedication, and a willingness to learn. He actively participated in assigned tasks and showed good teamwork and professional behavior throughout the internship.

We appreciate his efforts and wish him success in future academic and professional endeavors.

Date: 13-04-2026

Certified by :

LOGIC WAVES TECH ★ MO.7041502002 ★

Bharat Shrivastav

CEO

Logic Waves Tech

This certificate is issued as part of Logic Waves Tech's industry skill development initiative.

7041502002

info@logicwavestech.com

www.logicwavestech.com

Office No. 20, first floor, balaji arcade, near mulad chowk, Kim, 394110

PMMS CERTIFICATE

DECLARATION

We hereby declare that the Internship report submitted along with the Internship entitled URBAN-STAY submitted in fulfilment for the degree of Bachelor of Engineering in **Computer-Engineering** to Gujarat Technological University, Ahmedabad, is a Bonafide record of original project work carried out by me at **Logic Waves Tech** under the supervision of and Internal Guide **Prof. Shreyas Patel** and that no part of this report has been directly copied from any students reports or taken from any other source, without providing due reference.

Name of the Student	Sign of Student
Rathod Harsh Indravadan	_____

ACKNOWLEDGEMENT

I sincerely thank our respected Head of Department, Dr. Namrata Shroff, and my internal guide, Prof. Shreyas Patel, for their constant support and guidance. Their valuable insights and encouragement greatly contributed to making this internship a meaningful and enriching experience.

Furthermore, I extend profound gratitude to the leadership team of Logic Waves Tech Solution, including CEO/founder Mr. Kruti Chaudhary and mentor Mr. Bharat Shrivastav, and the entire workforce for their invaluable assistance and support throughout the project's execution. Their guidance and encouragement provided us with invaluable insights and experiences, shaping us into more competent professionals as we embark on our future endeavours.

Moreover, I am deeply grateful to Gujarat Technological University for recognizing the significance of incorporating Internship as a compulsory subject, and uniquely positioning it as the sole subject in the final semester. This innovative approach provides us with an early immersion into the industrial environment and practical experience.

In conclusion, I am deeply indebted to my parents, friends, and cherished family members for their unwavering support and boundless inspiration, which fueled our dedication towards the successful completion of our internship endeavors.

Rathod Harsh Indravadan

(230143107017)

ABSTRACT

During the internship, my primary objective was to gain exposure to the professional work environment and understand industry operations. As students, we often lack direct experience with real-world workflows, but this opportunity allowed me to bridge that gap. I am sincerely grateful to GTU and our college for providing us with this invaluable platform to engage in Realtime office work.

I chose full stack Application Development as my domain, where I underwent training to a project. Throughout the training, I gained hands-on experience with various technologies, including ReactJS, MongoDB, JavaScript, and Tailwind CSS. During the internship, I worked extensively with different APIs, learning how they enhance functionality and provide seamless user experiences. Beyond core development, I was involved in multiple aspects of the project, such as debugging critical issues, implementing advanced filtering mechanisms, enabling dynamic object movement features, ensuring seamless database connectivity, and enhancing user engagement through intuitive design elements.

LIST OF FIGURES

Fig 1.1 SDLC Phases 05

Fig 3.1 Gantt Chart of Internship 16

Fig 5.1 Use Case diagram for Admin 29

Fig 5.2 Use Case diagram for User 29

Fig 5.3 Activity Diagram of User 30

Fig 5.4 Activity Diagram of User 31

Fig 5.5 Activity Diagram of Admin 32

Fig 5.6 E-R Diagram 33

Fig 5.7 Request appointment 34

Fig 5.8 New User info 34

Fig 5.9 Seller Role 35

Fig 5,10 Buyer Role 35

Fig 6.1 Home page 1 37

Fig 6.2 Home page 2 37

Fig 6.3 Admin Login 41

Fig 6.4 Admin Dashboard 41

Fig 6.5 For Selling Property 42

Fig 6.6 Property Details 42

Fig 6.7 Property Card 43

Fig 6.8 Home Interior 43

LIST OF TABLES

Table 7.1: Admin Login 45

Table 7.2: User Forgot password 46

Table 7.3: User Click Button 47

TABLE OF CONTENTS

TITLE OF PROJECT
CERTIFICATE
INDUSTRY LETTER HEAD
PMMS CERTIFICATE
DECLARATION
ACKNOWLEDGEMENT
ABSTRACT
LIST OF FIGURES
LIST OF TABLES
TABLE OF CONTENTS

CHAPTER 1: OVERVIEW OF THE COMPANY 1
1.1 HISTORY OF THE COMPANY 1
1.2 SCOPE OF WORK 1
1.3 ORGANIZATION CHART 3
CHAPTER 2: OVERVIEW OF THE ORGANIZATION 3
2.1 WORK DETAILS OF THE DEPARTMENTS 4
2.2 TECHNICAL SPECIFICATION 5
2.3 SEQUENCE OF OPERATIONS IN THE ORAGANIZATION FOR SOFTWARE DEVELOPMENT PROCESS 5
CHAPTER 3 INTRODUCTION TO INTERNSHIP 6
3.1 INTERNSHIP SUMMARY 7
3.2 PURPOSE 7
3.3 OBJECTIVE 8
3.4 SCOPE OF INTERNSHIP 9
3.5 TECHNOLOGY AND LITERATURE REVIEW 9
3.5.1 FUNCTIONALITY AND ADVANTAGE 10
3.6 INTERNSHIP PLANNING 11
3.6.1 INTERNSHIP DEVELOPMENT APPROACH AND JUSTIFICATION 12
3.6.2 INTERNSHIP EFFORT AND TIME, COST ESTIMATION 13
3.6.3 ROLES AND RESPONSIBILITIES 14
3.7 INTERNSHIP SCHEDULING 15

CHAPTER 4 SYSTEM ANALYSIS 16

4.1 STUDY OF CURRENT SYSTEM 17

4.2 PROBLEM AND WEAKNESSES OF CURRENT SYSTEM 18

4.3 REQUIREMENTS OF NEW SYSTEM 18

4.4 SYSTEM FEASIBILITY 19

4.4.1 DOES THE SYSTEM CONTRIBUTE TO THE OVERALL OBJECTIVES OF THE ORGANIZATION? 19

4.4.2 CAN THE SYSTEM BE IMPLEMENTED USING THE CURRENT TECHNOLOGY AND WITHIN THE GIVEN COST AND SCHEDULE 20

4.4.3 CAN THE SYSTEM BE INTREGRATED WITH OTHER SYATEM WHICH ARE ALREADY PLACE 21

4.5 ACTIVITY / PROCESS IN NEW SYSTEM / PROPOSED SYSTEM 21

4.6 FEATURES OF NEW SYSTEM / PROPOSED SYSTEM 22

4.7 LIST MAIN MODULES / COMPONENTS / PROCESSES / TECHNIQUES OF NEW SYSTEM 22

4.8 SELECTION OF HARDWARE / SOFTWARE / ALGORITHMS / METHODOLOGY / TECHNIQUES / APPROACHES AND JUSTIFICATION: 23

CHAPTER 5 SYSTEM DESIGN: 27

5.1 SYSTEM DESIGN AND METHODOLOGY 27

5.2 PROCESS DESIGN 32

CHAPTER 6 IMPLEMENTATION 34

6.1 IMPLEMENTATION PLATFORM 35

6.2 PROCESS 35

6.3 FINDING / RESULTS / OUTCOMES 40

CHAPTER 7 TESTING 44

7.1 TESTING PLAN 45

7.2 TEST RESULTS AND ANALYSIS 46

CHAPTER 8 CONCLUSION AND DISCUSSION 47

8.1 OVERALL ANALYSIS OF INTERNSHIP 47

8.3 PROBLEM ENCOUNTERED AND POSSIBLE SOLUTIONS 47
8.4 SUMMARY OF INTERNSHIP 48
8.5 LIMITATION AND FUTURE ENHANCEMENT 48
CHAPTER 9 : REFERENCES 50

CHAPTER: 1 OVERVIEW OF THE COMPANY

1.1 HISTORY OF THE COMPANY:

▪ Logic Waves Tech was established in early 2024 in Kim, Gujarat, by a team of four dedicated engineers and digital strategists.

▪ The company is at the forefront of digital engineering in the Kim-Olpad region, delivering high-performance solutions in web architecture, e-commerce, and specialized IT training.

▪ A core mission of the studio is to bridge the gap between local talent and global standards, providing hands-on learning through intensive 8–12 week on-site internship tracks

▪ Expertise includes Full-Stack Development (React.js, Next.js, Node.js, .NET), E-Commerce Engineering (Shopify Plus, WooCommerce), and Generative AI (RAG pipelines and LLM integration).

▪ The firm provides customized digital growth strategies, combining technical SEO structures with high-ROI Google Ads management to scale brands effectively.

▪ With a focus on precision and scalability, the company delivers cost-effective, senior-level tech solutions that empower local businesses to compete in the global digital economy.

1.2 SCOPE OF WORK:

- The company offers end-to-end digital services, from system blueprints to deployment and growth management. Their main services include:
- **Platform & Web Architecture**: Building web applications and scalable architectures with frameworks like React.js, Next.js, Node.js, and ASP.NET Core.
- **E-Commerce Solutions**: Developing storefronts on platforms such as Shopify Plus and WooCommerce, including custom Liquid development and API integrations.
- **Generative AI Systems**: Integrating Large Language Models (LLMs) and custom RAG pipelines into business workflows using Python and tools like OpenAI or Gemini.
- **Growth & Marketing:** Managing technical SEO structures and high-ROI Google Ads (PPC) pipelines to improve search visibility and lead generation.
- **UI/UX Design Services:** Offering prototyping, wireframing, and user testing for enhanced user experiences.

- **Cybersecurity Services:** Implementing security measures to protect client data and applications.
- **Live Project Experience:** Providing opportunities to work on real-world projects for hands-on learning.

1.3 CAPACITY OF ORGANIZATION:

As an organization, our capacity is characterized by our focused team of 10 dedicated employees who collectively drive our projects forward with expertise and efficiency. Company operates from a tech-equipped facility with high-performance systems, secure servers, and advanced development and testing tools. The infrastructure ensures efficient project execution, data security, and seamless software development, supporting the company's commitment to delivering high-quality IT solution.

CHAPTER: 2 OVERVIEWS OF THE ORGANIZATION

2.1 WORK DETAILS OF THE DEPARTMENTS:

1. **Planning and Requirement Gathering:**
   - Understanding the user's needs and objectives for the analysis.
   - Gathering requirements, including features, functionalities, and design preferences.
   - Defining project scope, timelines etc..
2. **Coding:**
   - Data collected
   - Applied pre-processing on dataset
   - Data visualized
   - Model training
   - Model testing
   - Gain insights from the model's results
   - Forecasted values
   - Stored results

2.2 TECHNICAL SPECIFICATION:

This part of chapter includes various departments of company.

1. **Data Science:** The growing demand for data scientists across industries presents exciting career opportunities in the digital age.
2. **AI and Machine Learning:** AI and ML simulate human intelligence, enabling computers to learn from data and revolutionize technology**.**
3. **MERN Stack:** It enables seamless data flow, efficient server-side handling, and dynamic user interfaces.
4. **Full stack (Frameworks (Django, Flask, MVC, Laravel)**): By leveraging full stack frameworks, developers can handle database integration, server-side logic, and user interface for comprehensive solutions.

5. **Mobile App (Android & Cross Platform):** By combining Android and iOS app development using Flutter, users can reach a broader client/customer base across multiple platforms while delivering engaging and seamless mobile experiences.

6. **Database (NoSQL):** MongoDB's powerful features and horizontal scalability through sharding make it popular in applications requiring flexibility and scalability.

2.3 SEQUENCE OF OPERATIONS IN THE ORAGANIZATION FOR SOFTWARE DEVELOPMENT PROCESS:

Following are the sequence of operations in the organization for software development process.

SDLC Phases

The entire SDLC process divided into the following stages:

Fig 2.1 SDLC Phases Phase1:

Requirement collection and analysis:

The requirement is the first stage in the SDLC process. It is conducted by the senior team members with inputs from all the stakeholders and domain experts in the industry. Planning for the quality assurance requirements and recognition of the risks involved is also done at this stage.

This stage gives a clearer picture of the scope of the entire project and the anticipated issues, opportunities, and directives which triggered the project. Requirements Gathering stage need teams to get detailed and precise requirements. This helps companies to finalize the necessary timeline to finish the work of that system.

Phase 2: Feasibility study:

Once the requirement analysis phase is completed the next step is to define and document software needs. This process conducted with the help of 'Software.

Requirement Specification' document also known as 'SRS' document. It includes everything which should be designed and developed during the project life cycle.

Phase3: Design

In this third phase, the system and software design documents are prepared as per the requirement specification document. This helps define overall system architecture. This design phase serves as input for the next phase of the model.

Phase4: Coding

Once the system design phase is over, the next phase is coding. In this phase, developers start build the entire system by writing code using the chosen programming language. In the coding phase, tasks are divided into units or modules and assigned to the various developers. It is the longest phase of the Software Development Life Cycle process. In this phase, Developer needs to follow certain predefined coding guidelines. They also need to use programming tools like compiler, interpreters, debugger to generate and implement the code.

Phase5: Testing

Once the software is complete, and it is deployed in the testing environment. The testing team starts testing the functionality of the entire system. This is done to verify that the entire application works according to the customer requirement.

During this phase, QA and testing team may find some bugs/defects which they communicate to developers. The development team fixes the bug and sends back to QA for a re-test. This process continues until the software is bug-free, stable, and working according to the business needs of that system.

Phase6: Installation/Deployment

Once the software testing phase is over and no bugs or errors left in the system then the final deployment process starts. Based on the feedback given by the project manager, the final software is released and checked for deployment issues if any.

Phase7: Maintenance

Once the system is deployed, and customers start using the developed system, following 3 activities occur

- Bug fixing - bugs are reported because of some scenarios which are not tested at all.
- Upgrade-Upgrading the application to the newer versions of the Software.
- Enhancement- Adding some new features into the existing software.

CHAPTER: 3 INTRODUCTION TO INTERNSHIP

3.1 INTERNSHIP SUMMARY:

- During my internship with Logic Waves tech Company, I had the opportunity to work on a variety of projects and gain practical experience in MERN Technology. Throughout the internship, I worked closely with senior developers and project managers to learn about the company's development process and best practices in the industry.
- One of the highlights of my internship was working on a team project to develop a custom website for a client. I was responsible for implementing the website's Find Agent Page using Tailwind and React, and collaborating with other team members to ensure seamless integration of all website features.
- Overall, the internship provided me with valuable experience in MERN Technology, improved my technical skills, and helped me gain an understanding of the business side of web development. I also developed communication skills by working with team members. I am grateful for the opportunity to have worked with Logic Waves tech Company and look forward to applying my skills in future web development projects.

3.2 PURPOSE:

- The purpose of pursuing an internship in MERN (MongoDB, Express.js, React.js, Node.js) technology is to gain practical, hands-on experience in developing full stack web applications.
- Through this internship, I aim to deepen my understanding of each component of the MERN stack and how they interact to create dynamic and efficient web applications. Additionally, I seek to enhance my skills in front-end development using React.js, back-end development with Node.js and Express.js, and database management with MongoDB.
- By working on real-world projects under the guidance of experienced professionals, I aim to sharpen my problem-solving abilities, improve my coding practices, and build a strong foundation for a career in web development. Furthermore, I aspire to contribute effectively to the team, collaborate with peers, and learn from their experiences, ultimately becoming a proficient MERN stack developer capable of delivering high- quality solutions in the industry.

3.3 OBJECTIVE:

1. **Understanding the Stack:** Gain a comprehensive understanding of the MERN stack, including its individual components (MongoDB, Express.js, React.js, Node.js), their interactions, and how they contribute to building dynamic web applications.
2. **Hands-on Development Experience:** Acquire practical experience by actively participating in the development process, working on real-world projects or tasks under the guidance of experienced mentors.
3. **Backend Development Proficiency**: Develop proficiency in backend development using Node.js and Express.js, including building RESTful APIs, handling authentication, authorization, and data manipulation.
4. **Frontend Development Skills:** Enhance skills in frontend development using React.js, focusing on building interactive user interfaces, managing state efficiently, and integrating with backend APIs.
5. **Database Management:** Learn database management concepts and practices using MongoDB, including schema design, querying, indexing, and ensuring data integrity.
6. **Problem-solving and Troubleshooting:** Develop problem-solving skills by encountering and resolving technical challenges encountered during the development process, such as debugging code, optimizing performance, and addressing security concerns.
7. **Collaborative Development Practices:** Gain experience working in a collaborative development environment, including version control using tools like Git, participating in code reviews, and adhering to coding standards and best practices.
8. **Continuous Learning and Adaptation:** Foster a mindset of continuous learning and adaptation to keep up with the rapidly evolving technologies and best practices within the MERN stack and the broader field of web development.
9. **Communication and Teamwork:** Improve communication and teamwork skills by effectively communicating ideas, collaborating with team members, and contributing positively to the overall project objectives.
10. **Project Management and Time Management:** Develop project management and time management skills by planning and organizing tasks effectively, prioritizing work based on

project requirements and deadlines, and delivering high-quality results within stipulated timelines.

3.4 SCOPE OF INTERNSHIP:

The scope of a MERN (MongoDB, Express.js, React.js, Node.js) technology internship encompasses a wide range of opportunities and experiences aimed at equipping interns with valuable skills and knowledge in modern web development. Here's how I would phrase it:

1. **Comprehensive Skill Development:** The Internship offers a comprehensive platform for interns to develop expertise in all aspects of the MERN stack, covering both fronted and backend development. This includes mastering technologies like MongoDB for database management, Express.js and Noe.js for backend development, and React.js for frontend user interface creation.
2. **Real-world Project Experience:** Interns get hands-on experience by working on real-world projects under the guidance of experienced mentors. These projects provide a practical understanding of how to apply MERN stack technologies to solve complex problems and develop scalable web applications.
3. **Industry-Relevant Knowledge:** The internship is designed to impart knowledge that is directly relevant to the needs of the industry. Interns learn about the latest trends, best practices, and tools used in MERN stack development, ensuring they are well-prepared to meet the demands of the job market upon completion of the program.
4. **Networking Opportunities:** Interns have the opportunity to network with professionals in the field of web development, including mentors, fellow interns, and industry experts. This networking can lead to valuable connections, mentorship opportunities, and potential job prospects in the future.
5. **Career Advancement:** By completing the internship, interns enhance their employability and open up new career opportunities in web development. The skills and experience gained during the internship make interns highly sought after by employers looking for proficient MERN stack developers.
6. **Personal Growth**: Beyond technical skills, the internship also fosters personal growth by encouraging interns to develop soft skills such as communication, teamwork, problem

solving, and time management. These skills are essential for success in any professional setting and contribute to the overall growth and development of interns.

7. **Flexibility and Adaptability:** The scope of the internship is flexible, allowing interns to tailor their learning experience based on their interests, strengths, and career goals. Whether focusing on frontend development, backend development, or full-stack development, interns have the flexibility to explore areas that align with their aspirations.

Overall, the scope of a MERN technology internship is vast and offers interns a dynamic learning environment where they can grow both personally and professionally while acquiring valuable skills that are in high demand in the ever-evolving field of web development.

3.5 TECHNOLOGY AND LITERATURE REVIEW:

Front-end technologies: HTML, CSS(Tailwind), JavaScript, and React.js

- HTML (Hypertext Markup Language) is the standard markup language for creating web pages and web applications.
- CSS (Cascading Style Sheets) is used to describe the presentation of HTML documents.
- JavaScript is a programming language used to create interactive web pages and web applications.
- React is a JavaScript library used for building user interfaces. Back-end technologies:
- Node.js is an open-source, cross-platform JavaScript runtime environment used for server-side programming.
- Express is a web application framework used for building web applications and APIs.
- MongoDB is a document-based NoSQL database used for storing data. Tools and frameworks:
- Git is a version control system used for tracking changes in code.
- Visual Studio Code is an open-source code editor used for writing, testing, and
- debugging code.
- Postman is a tool used for testing APIs.

3.5.1 Functionality and Advantages:

The MERN stack offers several key advantages:

- Full-stack JavaScript development: With JavaScript being the primary language for both frontend and backend development, MERN facilitates a unified and seamless development experience.
- Rapid prototyping: The modular structure of MERN components enables rapid prototyping and iterative development, speeding up the overall development process.
- Scalability: MERN applications can easily scale to accommodate growing user bases and evolving business requirements, thanks to the scalability features of MongoDB and Node.js.
- Community support and ecosystem: The MERN stack benefits from a vibrant community and a rich ecosystem of libraries, frameworks, and tools, enhancing developer productivity and collaboration. Literary Perspective:
- Beyond its technical merits, MERN technology also holds literary significance:
- Narrative structure: The MERN stack can be likened to the narrative structure of a story, with MongoDB serving as the foundation (setting), Express.js as the plot framework, React as the characters and interactions, and Node.js as the storyteller (server).
- User experience (UX): Just as literature aims to engage and captivate readers, MERN applications strive to deliver compelling user experiences through intuitive interfaces and seamless interactions.
- Evolution and adaptation: Like literary genres that evolve over time, MERN technology continues to evolve and adapt to changing trends and demands, ensuring its relevance and longevity in the fast-paced digital landscape.

3.6 INTERNSHIP PLANNING:

1. **Understanding the Basics:**

- Familiarize yourself with the MERN stack components: MongoDB (a NoSQL database), Express.js (a web application framework for Node.js), React.js (a JavaScript library for building user interfaces), and Node.js (a JavaScript runtime).
- Learn about the role of each component in a MERN application and how they interact with each other.

2. **Setting Up Your Development Environment:**

- Install necessary software and tools such as Node.js, MongoDB, a code editor (like Visual Studio Code), and any additional packages or extensions required for development.
- Set up a version control system like Git and create a repository to track your code changes.

3. **Learning MongoDB:**

- Understand the basics of MongoDB, including its document-oriented data model, collections, documents, and queries.
- Practice CRUD (Create, Read, Update, Delete) operations in MongoDB.
- Explore more advanced topics such as indexing, aggregation, and schema design.

4. **Mastering Express.js:**

- Learn how to create RESTful APIs using Express.js.
- Understand routing, middleware, error handling, and security considerations in Express.js applications.
- Practice integrating Express.js with MongoDB to perform CRUD operations.

5. **Exploring React.js:**

- Learn about React components, JSX syntax, state management, and handling user events.
- Gain proficiency in building user interfaces with React.js.
- Explore React hooks and context API for state management and component communication.

6. **Understanding Node.js:**

- Deepen your knowledge of Node.js and its event-driven architecture.

- Learn about asynchronous programming, event loop, and Node.js core modules.
- Practice building server-side applications and APIs with Node.js.

7. **Integrating MERN Stack:**

- Combine your knowledge of MongoDB, Express.js, React.js, and Node.js to build a full-stack MERN application.
- Learn how to connect your React frontend to Express backend using RESTful APIs or GraphQL.
- Implement features like user authentication, authorization, and data validation.

8. **Testing and Debugging:**

- Learn testing techniques for MERN applications, including unit testing, integration testing, and end-to-end testing.
- Explore debugging tools and strategies for identifying and fixing issues in your codebase.

9. **Deployment and DevOps:**

- Understand the deployment process for MERN applications on platforms like Heroku, AWS, or Digital Ocean.
- Explore continuous integration and continuous deployment (CI/CD) pipelines to automate the deployment process.

10. **Project Work and Portfolio Building:**

- Work on real-world projects or contribute to open-source projects to apply your skills and gain practical experience.
- Build a portfolio showcasing your MERN development projects, including code samples, project descriptions, and deployment links.

3.6.1 INTERNSHIP DEVELOPMENT APPROACH AND JUSTIFICATION:

- During my internship focused on the MERN stack (MongoDB, Express.js, React.js, Node.js), the program followed a straightforward yet effective approach. It began with a clearly outlined learning path that guided me from fundamental concepts to more advanced topics. This step-by-step progression helped me grasp each concept

thoroughly before moving on, making the overall learning experience more manageable and less overwhelming.

- Next, the internship program prioritized hands-on learning through project-based work. I had the opportunity to dive into real-world applications, which allowed me to apply my theoretical knowledge in practical scenarios. This approach not only reinforced my understanding but also sharpened my problem-solving skills and prepared me for the real-world challenges I may face in my career.
- Additionally, the internship emphasized mentorship and feedback throughout the program. I was paired with a mentor who provided valuable guidance and support, creating a positive and conducive learning environment. Regular code reviews and constructive feedback sessions helped me improve my skills and refine my coding practices, ensuring continuous growth and development.

3.6.2 INTERNSHIP EFFORT AND TIME AND COST ESTIMATION:

The effort, time, and cost estimation for my internship in a Urban-stay project depended on several factors, including the scope of the project, the duration of the internship, and the level of involvement required from me as an intern. Several key considerations were taken into account while estimating these aspects, ensuring that the internship was planned efficiently and aligned with the project's overall objectives.

- Scope of the Project: The scope of the project affected how much time and effort I needed to put in. Since it was a big project, it took more time and work to complete my tasks.
- Duration of the Internship: The length of my internship also affected how much time and effort I needed to give. Since it was a longer internship, I got more chances to learn and build my skills, but it also meant putting in more time and staying committed throughout.
- Availability of resources: The availability of resources, such as equipment, software, and personnel, can impact the time and cost of an internship. A lack of resources may result in a longer duration or higher cost of the internship.

- Skills and Experience of the Team: The skills and experience of the development team can impact the effort and time required. A team with more experience and skills can complete tasks more efficiently and effectively.
- Complexity of the Website: The complexity of the website affected the time and effort I had to put in. A website with more features and functionalities took more time to build and required extra attention to detail.
- Tools and Technologies Used: The tools and technologies used for website development affected the time and effort needed. Using modern tools and technologies helped speed up the development process and made tasks easier to complete.

Before working on the Urban-stay website development, my mentor assigned me the task of developing the Fashion Hub e- Commerce website. Based on the factors considered for the internship, an estimate for the effort and time required for this task in website development could range from 100-200 hours over a period of 3-6 months. This estimate includes planning, design, development, testing, and maintenance.

Cost estimation for an internship in Urban-stay website development can vary depending on the location, the complexity of the project, and the size of the development team. In general, the cost can range from $2,000-$5,000, which includes the cost of resources, software, and other development expenses.

In conclusion, effort and time estimation for an internship in Urban-stay website development can vary based on several factors. Proper planning, efficient use of resources, and effective project management can help ensure that the internship is completed within the estimated timeframe and budget.

3.6.3 ROLES AND RESPONSIBILITIES:

- Continue to learn my skills as a Full-Stack Intern
- Develop responsive web applications and present weekly progress updates to mentors and fellow interns
- Learn soft skills to be better eligible for real world interaction with client and colleagues.

- Developed habits of following industry standards and good practices for software development under the guidance of my mentor.
- Chose the right way to learn new things.
- Developing professional skills, including communication, teamwork, and problem-solving.
- Creating user interfaces using React.js with a focus on performance and usability.

3.7 INTERNSHIP SCHEDULING:

3.1 Gantt Chart of Internship

CHAPTER 4: SYSTEM ANALYSIS

4.1 STUDY OF CURRENT SYSTEM:

Examining a Urban-stay website for system analysis involves evaluating its components, functionalities, and interactions to understand its workflow and areas for improvement. Below are key aspects to consider:

1. System Overview:

- Identify the purpose and objectives of the Urban-stay website.
- Understand the target audience, including families, working professionals, children, and elderly patients.
- Define the scope of the system analysis to evaluate key digital functions like appointment booking, patient records, and billing.

2. Functionality:

- List and describe the main features of the website, such as appointment scheduling, service listings, patient form submission, treatment history access, and online consultation booking.
- Analysed the integration of automated appointment systems and patient support services, including reminders, confirmations, and follow-up notifications.

3. User Roles:

- Identify different user roles, including patients, dental staff, and administrators, along with their specific permissions and access levels within the system.
- Examine the user journey for each role, from initial appointment inquiry to treatment completion and post-visit follow-up. 4. Performance:
- Analyze the website's performance, including loading speed, server response time, and the efficiency of retrieving patient data and service information.
- Identify potential areas for optimization, such as streamlining database queries, enhancing caching mechanisms, and reducing page load times.

5. Mobile responsiveness:

- Evaluate how well the website adapts to different screen sizes and devices.
- Ensure that all features, including form submissions, payment processing, and customer support, function seamlessly across mobile and tablet devices.

4.2 Problem and Weaknesses of Current System

- **Poor user experience:**
  Complicated navigation, unclear instructions, and a cluttered interface can make it difficult for patients to book appointments or find relevant information. Slow page loading times and frequent system errors may frustrate users, potentially leading to missed appointments or patients abandoning the website altogether.
- **Ineffective Search Functionality:**
  A poor or non-existent search feature can make it difficult for users to quickly find specific services, treatment information, or clinic details, leading to a frustrating user experience and reduced engagement.
- **Security Concerns:**
  Inadequate protection of patient data, such as unsecured login systems or lack of encryption, can lead to breaches of sensitive medical information, violating privacy laws and damaging trust.
- **Lack of Mobile Optimization:**
  A website that isn't responsive or user-friendly on mobile devices can hinder patients from booking appointments, accessing services, or viewing contact details on the go, leading to lost engagement and lower satisfaction.

4.3 Requirement of New System

When developing the new Urban-Stay website, the system should be designed around the real needs of property seekers, owners, and administrators. The platform must prioritize accessibility, security, and a smooth user experience. Key requirements include:

- **User-friendly Interface:**
A clean, modern, and intuitive design that helps users easily search properties, explore services, schedule site visits, and access important information.

- **Data Security and Privacy:**
Strong encryption, secure authentication, and role-based access controls to protect user data, property records, and transaction details.

- **Online Booking and Scheduling:**
A seamless, real-time booking system for property visits/consultations, with instant confirmations and automated reminders.

• **Scalability and Performance:**
A fast-loading and scalable architecture that can handle growing traffic, listings, and feature expansion without performance issues.

• **Secure User Portal Access:**
A protected login area where users can manage profiles, track inquiries, view booking history, save favorite properties, and manage appointments.

4.4 System Feasibility:

Assessing the feasibility of the Urban-Stay website or user portal means checking whether it supports business goals and improves customer experience. The system should be practical, scalable, and valuable for both users and management.

4.4.1 Does the system contribute to the overall objectives of the organization?

• **Enhanced Online Presence:**
A modern, SEO-friendly website can improve Urban-Stay's visibility, attract more users, and build trust through a professional and informative platform.

• **Improved Customer Satisfaction:**
The system can improve customer satisfaction by allowing easy property search, quick inquiry submission, simple booking of site visits, automated reminders, and better communication.

• **Cost Reduction:**
The new system can reduce costs by automating routine tasks, lowering manual errors, reducing paperwork, and improving staff productivity.

• **Improved Operational Efficiency:**
Urban-Stay can improve efficiency by streamlining inquiries, managing property listings better, simplifying appointment scheduling, and organizing customer data in one place.

• **Stronger Brand Reputation:**
Urban-Stay can strengthen its reputation by providing reliable service, clear property information, transparent pricing, timely responses, and a smooth digital experience.

Overall, the Urban-Stay system should align with the organization's core goals by improving efficiency, reducing operational costs, increasing customer satisfaction, and supporting long-term growth. By automating key processes and improving user engagement, the platform can deliver a better experience while helping the business scale successfully.by delivering high-quality care, maintaining accurate patient records, ensuring timely appointments, providing transparent pricing, and leveraging advanced technology for superior dental treatments.

4.4.2 Can the system be implemented using the current technology and within the given cost and schedule constraints.

Yes, the Urban-Stay system can be implemented using current technologies and can be delivered within reasonable cost and time limits.

• **Use of Current Technology:**

Urban-Stay can be built with modern and widely used technologies such as React.js for the frontend, Node.js for the backend, and MongoDB/MySQL for data storage. These technologies are reliable, well-supported, and suitable for real-world business platforms.

• **React.js for Scalable Frontend:**

React.js is highly scalable and works well for small to large web applications. It supports reusable components, making the system easier to develop, maintain, and expand as Urban-Stay grows.

• **Fast and Efficient UI Performance:**

React.js uses a Virtual DOM, which improves rendering speed and gives users a smooth experience even when pages contain dynamic property listings and frequent updates.

• **Better Developer Productivity:**

With reusable UI components and a large developer community, React.js helps reduce development time and coding effort. This supports faster delivery within schedule constraints.

• **SEO and Visibility Support:**

React.js can be integrated with server-side rendering or SEO-friendly practices, helping Urban-Stay pages rank better in search engines and improve online visibility.

• **Node.js for Strong Backend:**

Node.js provides a high-performance backend and is suitable for handling many users at the same time. It is ideal for features like real-time inquiries, bookings, and notifications.

• **Non-blocking Architecture:**

Node.js uses a non-blocking I/O model, allowing the server to process multiple requests efficiently. This improves speed and supports heavy traffic without major slowdowns.

• **Easy Scalability for Growth:**

Node.js can scale horizontally across servers, which helps Urban-Stay manage future traffic growth and business expansion.

• **Faster Development with Pre-built Modules:**

Both React.js and Node.js have rich ecosystems with many pre-built libraries and modules. This reduces custom development effort, lowers cost, and helps complete the project on time.

Overall, the Urban-Stay portal is technically feasible with the current technology stack, and it can be developed within expected budget and schedule constraints if planned and managed properly.

4.4.3 Can the system be integrated with other systems which are already in place?

• The System is built by Node JS and React JS so that they can be integrated with any web app.

4.5 ACTIVITY / PROCESS IN NEW SYSTEM / PROPOSED SYSTEM

To develop a Urban-stay website, here are some activities that need to be undertaken:

➢ **Requirement Gathering and Analysis:**

This involves gathering and understanding the requirements of the new system from stakeholders. It includes defining features, functionality, and any constraints or limitations.

➢ **System Design:**

Once requirements are gathered, the system architecture and design are planned. This includes defining the overall structure of the system, the components/modules, and how they interact with each other.

- **Implementation/Coding:**

Developers write the code for the system based on the design specifications. This phase involves writing, testing, and integrating code to build the system.

- **Testing:**

Quality assurance engineers test the system to ensure it meets the requirements and functions correctly. This includes various types of testing such as unit testing, integration testing, and system testing.

- **Deployment:**

The system is deployed to a production environment where end users can access and use it. This may involve installation, configuration, and setup procedures.

- **Maintenance and Support:**

After deployment, the system requires ongoing maintenance and support to address issues, implement updates, and make enhancements as needed.

- **Documentation:**

Throughout the development process, documentation is created to describe the system's functionality, architecture, and how to use it. This documentation is important for developers, testers, and end-users.

- **Feedback and Iteration:**

Feedback from users and Patients is collected to identify areas for improvement. Based on this feedback, iterations of the development process may occur to enhance the system further.

These activities typically occur iteratively, especially in Agile methodologies where development is broken down into smaller, incremental phases called sprints. Each sprint involves a cycle of planning, development, testing, and review, allowing for continuous improvement and adaptation throughout the development lifecycle.

4.6 FEATURES OF NEW SYSTEM / PROPOSED SYSTEM

1. **User Authentication and Authorization**
   - Users can securely sign up, log in, and manage their accounts.
   - Role-based access can be provided for customers, property owners, and admins.
   - Mandatory details are collected during registration for better profile and inquiry management.
2. **Online Booking and Visit Scheduling**
   - Users can book property visits in real time based on available slots.
   - System sends automated booking confirmations and reminder notifications.
   - Users can reschedule or cancel bookings easily from their dashboard.
3. **Property and User Records Management**
   - Secure storage of property details, inquiry history, booking records, and user preferences.
   - Easy access to saved properties, past interactions, and account activity.
   - Admin can manage and update listings efficiently from a centralized panel.
4. **Billing and Payment Integration**
   - Support for secure online payments for featured listings, subscriptions, or service charges.
   - Automatic invoice generation and payment status tracking.
   - Smooth transaction handling to improve trust and convenience.
5. **Reporting and Analytics**
   - Dashboard insights on inquiries, bookings, popular locations, and user behavior.
   - Revenue and conversion reports for business monitoring.
   - Helps management make data-driven decisions for growth.
6. **Multi-Platform Accessibility**
   - Fully responsive design for desktop, tablet, and mobile devices.
   - Smooth and consistent user experience across platforms.
   - Easy access for customers, agents, and admins anytime, anywhere.

4.7 LIST MAIN MODULES / COMPONENTS / PROCESSES / TECHNIQUES OF NEW SYSTEM / PROPOSED SYSTEM

- **Web User Pages:(Customers)**

  - **Home Page:** Includes header, hero banner/slider, featured properties, key services, city highlights, and footer.

• **About Us Page:** Presents Urban-Stay's mission, values, and background to build trust with visitors.

• **Contact Us Page:** Allows users to contact support/admin through a form with name, phone/email, and message.

• **New User Onboarding Page:** Provides a short platform introduction and a simple registration form for new users.

• **Finance and Loan Guidance Page:** Shares basic home loan, rent budgeting, and payment planning suggestions.

• **Request a Property Visit Page:** Users submit name, phone, preferred date/time, and property details to schedule a visit.

• **Our Services Page:** Shows services such as buying, selling, renting, legal support, and home interior assistance.

• **Review and Testimonials Page:** Users can submit reviews and read feedback from other customers

- **Admin Pages:**
  - **User Management**: Add, edit, block, or remove users, agents, and administrators with role-based access control.
  - **Property Management:** Add, approve, update, or remove property listings and media.
  - **Booking Management:** View, confirm, reschedule, or cancel property visit requests.
  - **Reports and Analytics:** Generate reports on inquiries, bookings, revenue, and performance trends.

4.8 SELECTION OF HARDWARE / SOFTWARE / ALGORITHMS / METHODOLOGY / TECHNIQUES / APPROACHES AND JUSTIFICATION:

Selection of Hardware:

- **Computer (Development Machine):**

A desktop or laptop is required for designing, coding, and testing the Urban-Stay website. Recommended minimum specifications are:

- **Processor:** Intel Core i3 or i5
- **RAM:** At least 4 GB
- **Storage:** At least 256 GB SSD

• **Monitor:**

A high-resolution monitor helps with development and design work. A 24-inch or 27-inch monitor with at least 1920×1080 resolution is recommended.

• **Server:**

A reliable server is needed to host the website and backend services. Recommended server specifications are:

- **Processor**: Intel Xeon or equivalent
- **Storage:** At least 500 GB (HDD or SSD)
- **Bandwidth:** At least 100 Mbps
- **Network Equipment:** Routers, switches, and firewalls are required to ensure secure and stable internet connectivity for both development and hosting environments.

- **Backup System:** It's important to have a backup system in place to protect against data loss. You can use cloud-based backup services or an external hard drive to back up your website files regularly.
- **Development Tools:** You will need software tools such as an IDE, text editor, web browser, and version control system to develop and test your website.

Selection of Software:

- **Operating System:** You will need an operating system for both your development computer and server. Windows, macOS, or Linux are popular choices.
- **Web Server:** You will need a web server to host your website. Apache, Nginx, and IIS are popular web servers.
- **Database:** You will need a database management system to store and manage your website's data. MySQL, PostgreSQL, and MongoDB are popular databases. we are used MySQL database.
- **Server-Side Language:** You will need a server-side language to create dynamic content on your website. PHP, Python, and Node.js are popular choices. We are use Node.js.
- **Front-End Framework**: You can use a front-end framework to build the user interface of your website. React, and Angular are popular choices.
- **Text Editor or Integrated Development Environment (IDE**): You will need a text editor or IDE to write and edit your website's code. Visual Studio Code, Atom, and Sublime Text are popular choices.

Algorithms

Authentication:

- Implementing algorithms for user authentication using libraries like Passport.js or JSON Web Tokens (JWT).
- Encrypting user passwords using algorithms like bcrypt.

Database Operations:

- Implementing algorithms for CRUD (Create, Read, Update, Delete) operations on MongoDB using Mongoose or native MongoDB drivers.
- Designing efficient database schemas and indexes for optimal performance.

API Design:

- Designing RESTful APIs for interacting with the backend using algorithms for routing and middleware in Express.js.
- Implementing algorithms for handling requests, parsing request bodies, and sending appropriate responses.

Frontend Logic:

- Implementing algorithms for client-side logic using JavaScript within React components.
- Utilizing algorithms for state management (e.g., Redux, Context API) and asynchronous operations (e.g., async/await, Promises).

Data Fetching:

- Using algorithms like Axios or Fetch API to fetch data from the server and update the UI accordingly.
- Implementing algorithms for handling loading states, errors, and caching data where necessary.

Real-time Communication:

- Implementing algorithms for real-time communication between client and server using technologies like Web Sockets (e.g., socket.io) or server-sent events.

Performance Optimization:

- Applying algorithms for optimizing frontend and backend performance, such as code splitting, lazy loading, server-side rendering, and database query optimization.

Methodology:

Agile Methodology:

- Agile methodology is a flexible and iterative approach to software development. It involves breaking down the development process into smaller chunks called sprints, with each sprint delivering a working product increment. The agile approach emphasizes collaboration between the development team and the stakeholders, including the Urban-stay company and its clients. This approach can be effective for Urban-stay website development since it allows for quick adaptation to changing requirements and feedback from users.

CHAPTER 5: SYSTEM DESIGN

5.1 SYSTEM DESIGN AND METHODOLOGY:

1. **Understanding System Design:**

- Team members and interns learn how to design scalable, secure, and efficient systems for modern web applications such as Urban-Stay.
- They understand architecture patterns, data modeling, and system components needed to build reliable and maintainable software.

2. **Methodologies for Development:**

- The team is introduced to common development approaches such as Agile, Scrum, and Waterfall.
- They learn how each methodology supports the software lifecycle, including planning, development, testing, deployment, and maintenance.
- For Urban-Stay, Agile is preferred because it supports quick feedback and continuous improvement.

3. **Practical Application:**

- Interns will apply system design principles and methodologies to real-world projects.
- They'll work on designing and implementing features within the context of a larger system, gaining hands-on experience in applying theoretical knowledge to practical scenarios.

4. **Collaborative Learning:**

- Interns will collaborate with mentors and peers to discuss system design decisions and methodologies used in their projects.
- They'll have opportunities to share insights, receive feedback, and learn from others experiences, fostering a collaborative and supportive learning environment.

- **Use case Diagram:**

Purpose: A use case diagram captures the functional view of a system from the user's perspective. It helps identify what the system should do and who interacts with it.

Main Purposes of a Use Case Diagram

- To gather system requirements clearly.

• To present an outside view of the system.

• To show interactions between actors and system functions.

Use Case Model

The use case model represents system requirements in a structured way. It is a communication tool between developers, users, and stakeholders to explain expected system behavior.

Actors

Actors are external entities that interact with the system. They can be:

• Human users (such as Customer, Admin, Agent)

• External systems (such as Payment Gateway, Notification Service)

• External devices or platforms

In diagrams, actors are usually shown as stick figures with role names.

Urban-Stay Context (Example Actors and Use Cases)

Typical Actors

• Visitor

• Registered User

• Property Owner/Agent

• Admin

• Payment Service

Typical Use Cases

• Register and log in

• Search and filter properties

• Book a property visit

• Submit inquiry

• Manage listings

• Approve or reject listings

- Process payments
- View reports and analytics

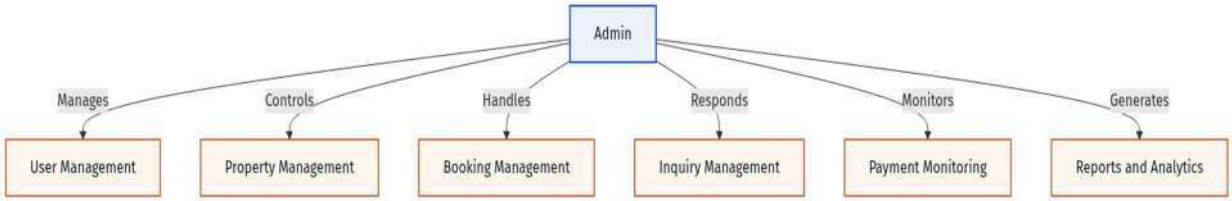

Fig 5.1 Use Case diagram for Admin

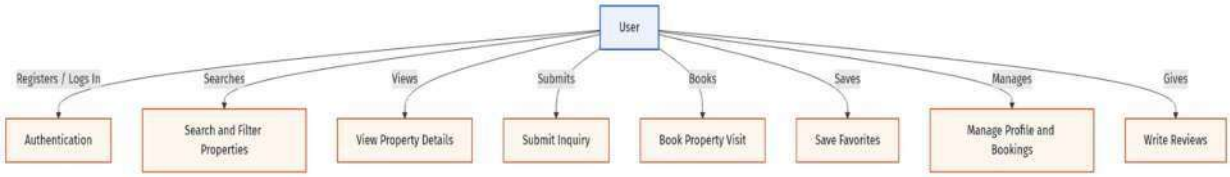

Fig 5.2 Use Case diagram for User

Activity Diagram:

- An activity diagram for Urban-Stay shows how users and the system interact step by step during key processes such as property search, inquiry, booking, and admin approval. It helps explain the workflow clearly, including actions, decision points, and parallel activities.
- In Urban-Stay, activity diagrams describe how different activities are coordinated to deliver services at different levels. For example, when a user books a property visit, the process includes searching properties, viewing details, selecting a slot, submitting booking data, checking availability, sending confirmation, and updating records. Some activities may run together, such as sending notifications while saving booking details. Identify candidate use cases, through the examination of business workflows.
    - Model workflows between/within use cases.
    - Model complex workflows in operations on objects.
    - Model in detail complex activities in a high-level activity Diagram.

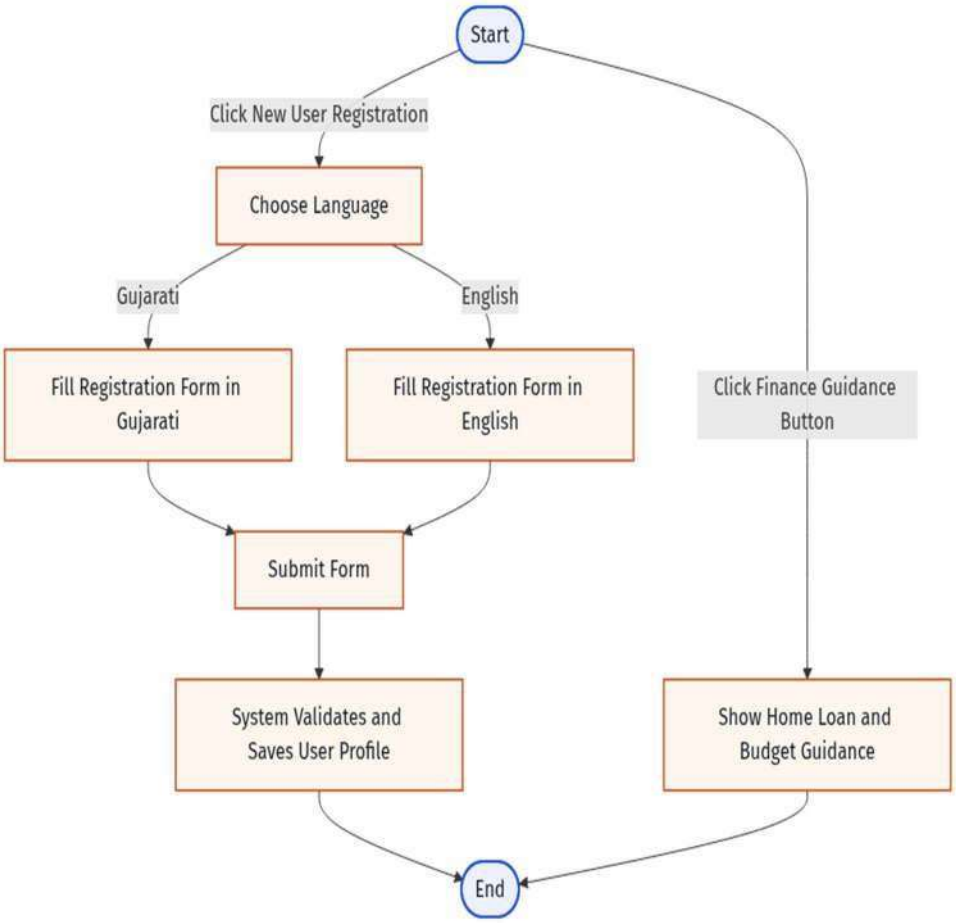

Fig 5.3 Activity Diagram of User (New user)

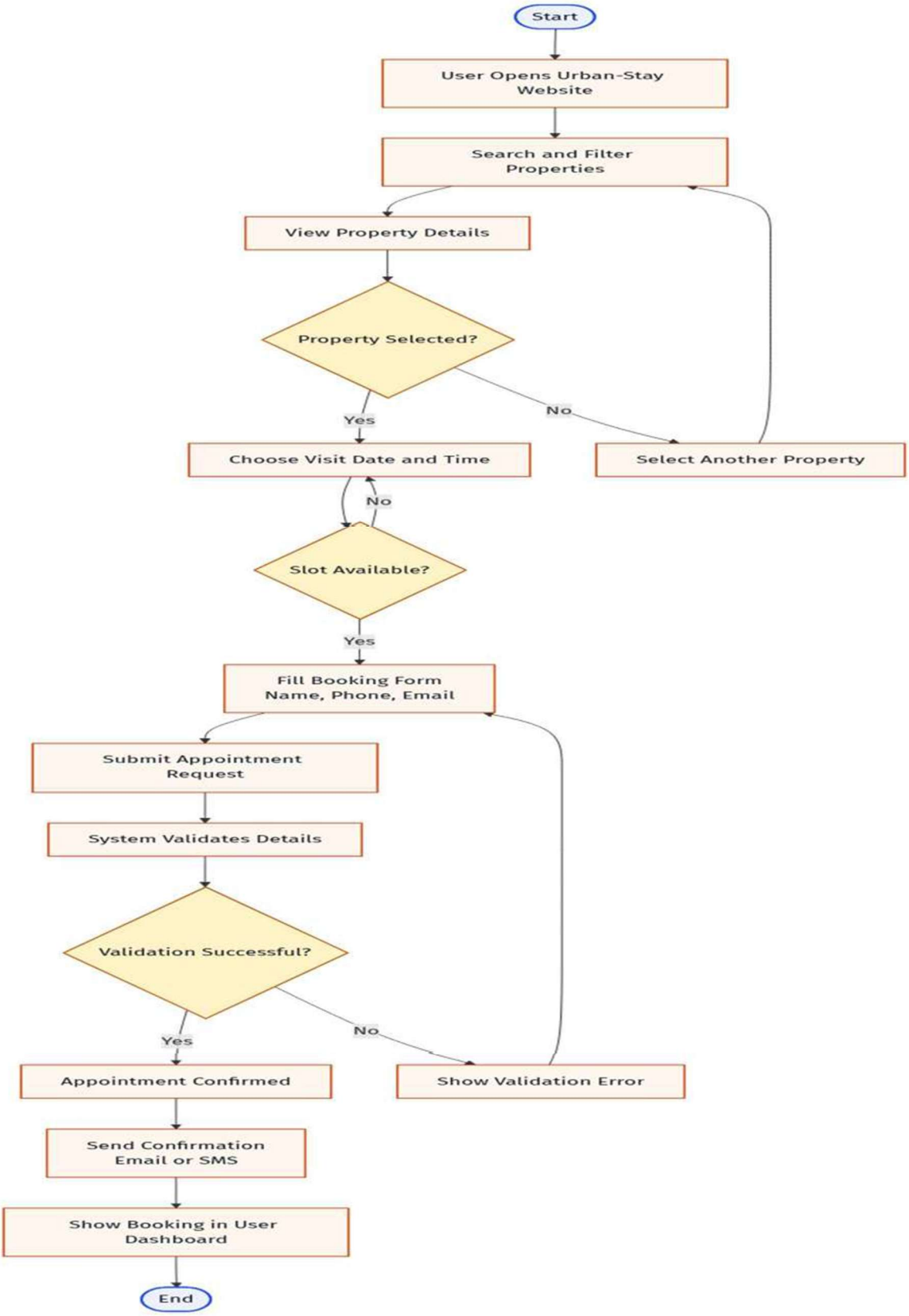

Fig 5.4 Activity Diagram of User (appointment book)

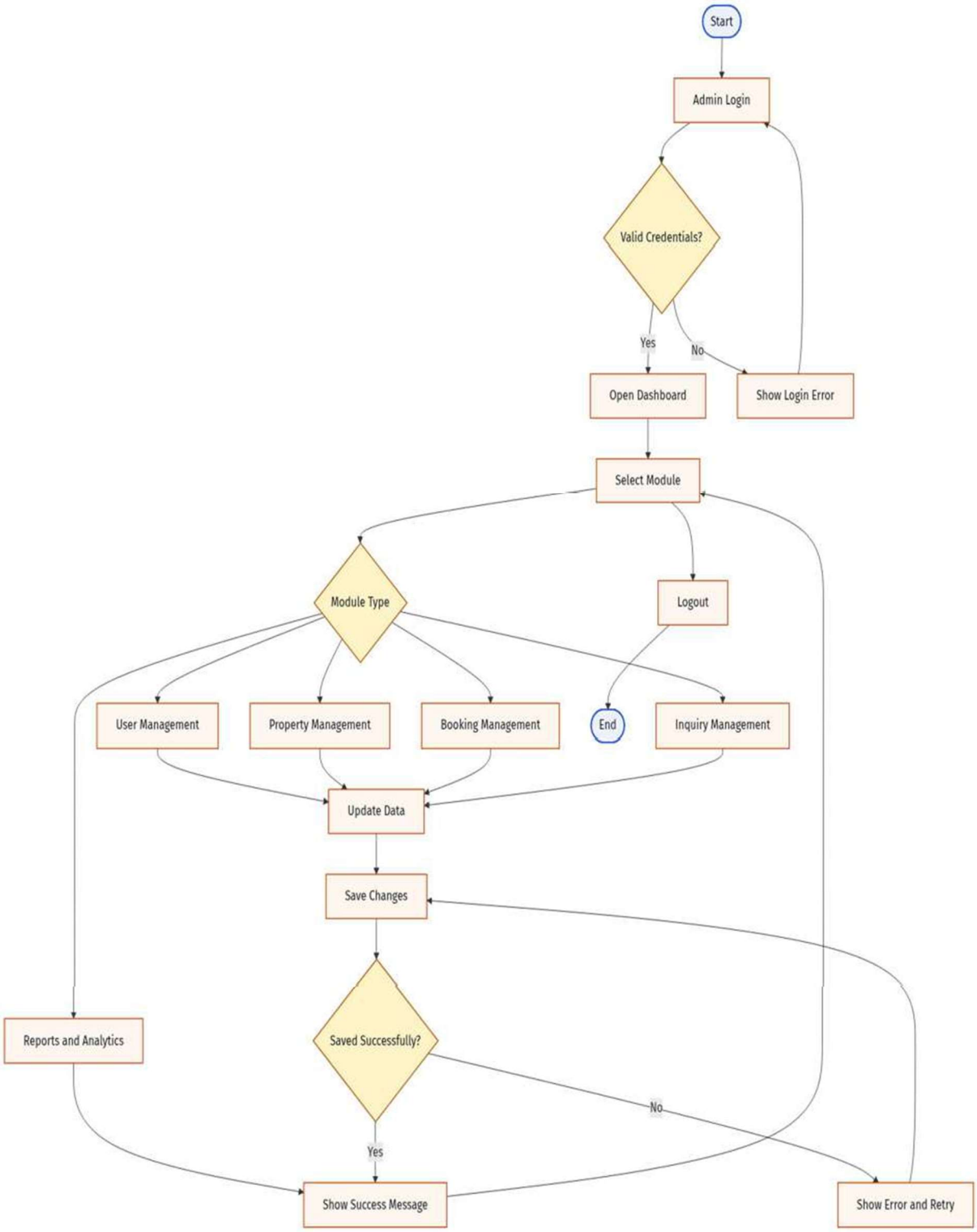

Fig 5.5 Activity Diagram of admin

E-R Diagram:

- ER model stands for an Entity-Relationship model. It is a high-level data model. This model is used to define the data elements and relationship for a specified system.
- It develops a conceptual design for the database. It also develops a very simple and easy to design view of data.
- In ER modelling, the database structure is portrayed as a diagram called an entity-relationship diagram.

For example, suppose we design a school database. In this database, the student will be an entity with attributes like address, name, id, age, etc. The address can be another entity with attributes like city, street name, pin code, etc and there will be a relationship between them.

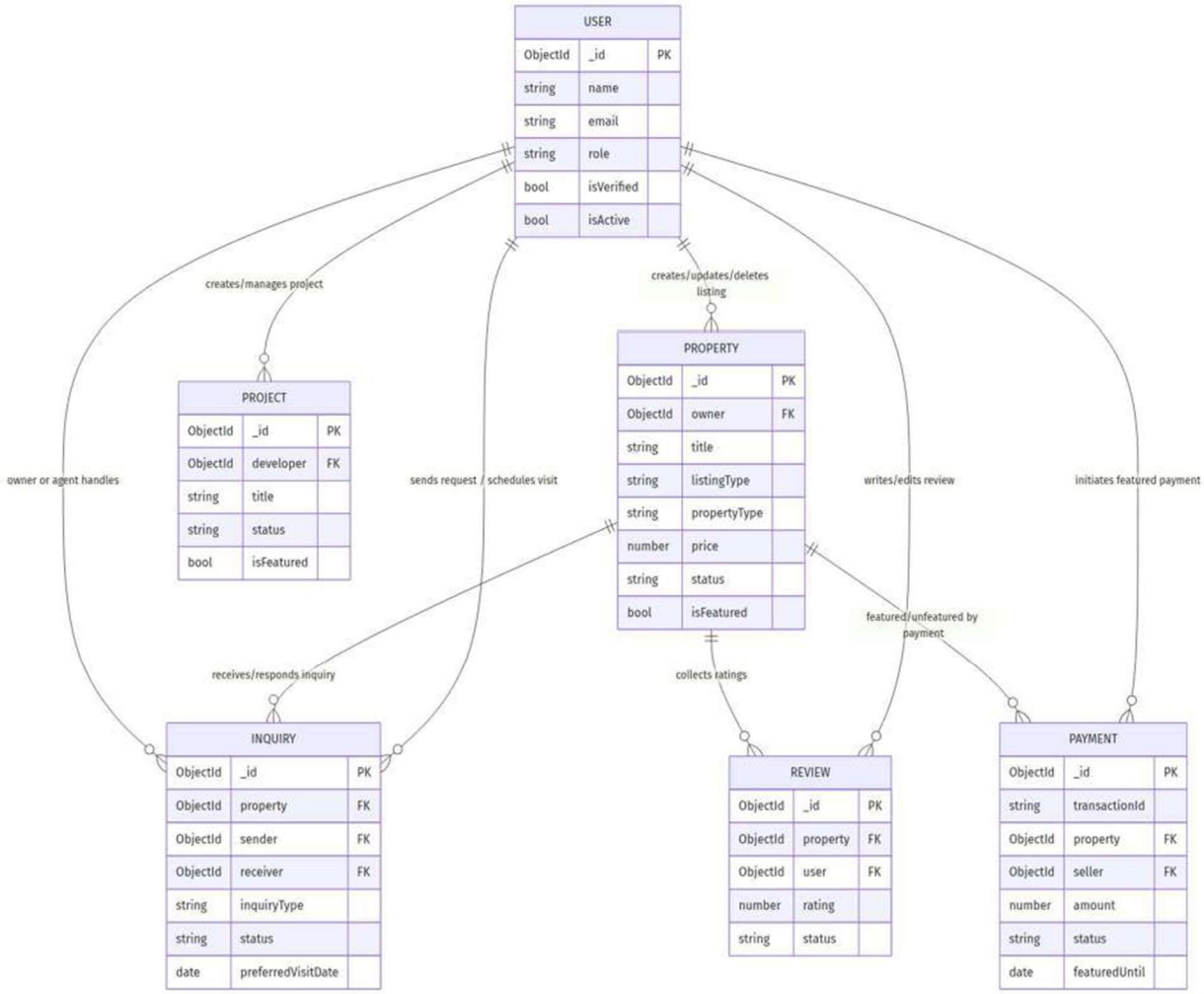

Fig 5.6 E-R Diagram

5.2 PROCESS DESIGN:

Column Name	MongoDB Type	Constraints	Description
_id	ObjectId	Primary Key, Auto Generated	Unique ID for each admin
name	String	Required	Admin full name
email	String	Required, Unique, Lowercase	Admin login email
password	String	Required, Hashed	Encrypted password
phone	String	Optional	Contact number
role	String	Enum, Value admin	System administrator role
isActive	Boolean	Default true	Admin account enabled status
isVerified	Boolean	Default false	Verification status
lastLogin	Date	Optional	Latest login time
createdAt	Date	Auto timestamp	Record creation time
updatedAt	Date	Auto timestamp	Last update time

Fig 5.7 Request appointment (DB for Admin)

Column Name	MongoDB Type	Constraints	Description
_id	ObjectId	Primary Key, Auto Generated	Unique ID for each user
name	String	Required, Trim, Min 2 chars	Full name of user
email	String	Required, Unique, Lowercase, Valid Email	User login email
password	String	Required, Min 6 chars, Hashed	Encrypted password
phone	String	Optional, 10-digit pattern	Contact number
role	String	Enum: user/seller/admin, Default user	User role in system
isVerified	Boolean	Default false	Account verification status
isActive	Boolean	Default true	Account active or blocked
registrationStatus	String	Enum: incomplete/complete/verified	Registration progress status
createdAt	Date	Auto timestamp	Record creation time
updatedAt	Date	Auto timestamp	Last update time

Fig 5.8 New User info

Column Name	MongoDB Type	Constraints	Description
_id	ObjectId	Primary Key, Auto Generated	Unique ID for each seller
name	String	Required	Seller full name
email	String	Required, Unique, Lowercase	Seller login email
password	String	Required, Hashed	Encrypted password
phone	String	Optional	Contact number
role	String	Enum, Value seller	Property seller role
companyName	String	Optional	Seller company or agency name
reraNumber	String	Optional	RERA registration number
isVerified	Boolean	Default false	Seller verification status
registrationStatus	String	Enum: incomplete/complete/verified	Onboarding progress
createdAt	Date	Auto timestamp	Record creation time

Fig 5.9 Seller-Role

Column Name	MongoDB Type	Constraints	Description
_id	ObjectId	Primary Key, Auto Generated	Unique ID for each buyer
name	String	Required	Buyer full name
email	String	Required, Unique, Lowercase	Buyer login email
password	String	Required, Hashed	Encrypted password
phone	String	Optional	Contact number
role	String	Enum, Value user	Buyer role in system
favorites	Array(ObjectId)	Ref Property	Saved property list
address.city	String	Optional	Buyer city
isVerified	Boolean	Default false	Account verification status
registrationStatus	String	Enum: incomplete/complete/verified	Onboarding progress
createdAt	Date	Auto timestamp	Record creation time

Fig 5.10 Buyer-Role

CHAPTER 6: IMPLEMENTATION

6.1 IMPLEMENTATION PLATFORM:

React js:

- ReactJS, also known as React, is a popular JavaScript library for building user interfaces. It is also referred to as a front-end JavaScript library. It was developed by Facebook and is widely used for creating dynamic and interactive web applications. In this article, we'll explore the key concepts of React.
- React operates by creating an in-memory virtual DOM rather than directly manipulating the browser's DOM. It performs necessary manipulations within this virtual representation before applying changes to the actual browser DOM. React is efficient, altering only what requires modification.
- React has a large ecosystem of tools and libraries that can be used to extend its functionality. It is often used in combination with other tools, such as Redux for state management and React Router for routing.

Node js:

- Node.js is a popular open-source JavaScript runtime built on Chrome's V8 JavaScript engine. It allows developers to build fast, scalable and efficient network applications using JavaScript both on the client and server-side.
- One of the main advantages of Node.js is its non-blocking I/O model, which allows for high concurrency and scalability. This means that Node.js can handle many concurrent connections without slowing down or blocking other requests.
- Node.js is also known for its ability to handle real-time applications, such as chat applications and online games, due to its event-driven architecture. Developers can use Node.js to create server-side applications, web applications, command-line tools, and even desktop applications.
- Node.js has a large ecosystem of packages and modules available through its package manager, npm. These packages can be used to extend the functionality of Node.js, making it easier for developers to build complex applications.

- Another advantage of Node.js is its ability to integrate with other technologies, such as databases, front-end frameworks, and cloud services. This makes it a versatile technology that can be used for a wide range of applications.

MongoDB:

- MongoDB is a NoSQL database management system developed by MongoDB Inc. It stores data in flexible, JSON-like documents (BSON) instead of fixed rows and columns. MySQL is one of the most recognizable technologies in the modern big data ecosystem. Often called the most popular database and currently enjoying widespread, effective use regardless of industry, it's clear that anyone involved with enterprise data or general IT should at least aim for a basic familiarity of My SQL.
- MongoDB is one of the most widely used modern databases in web and cloud applications. It is popular because it is easy to scale, works well with large and changing data, and fits modern development stacks such as Node.js and React.
- A database is an organized collection of data. It can store anything from simple user details to large business records. In MongoDB, data is grouped into collections, and each record is stored as a document. Unlike traditional relational databases, MongoDB does not require a strict table schema, so fields can vary between documents when needed. This flexibility makes it useful for fast development, real-time applications, and systems where data structure may evolve over time.

REST API:

- REST (Representational State Transfer) is an architectural style for building web services that allow different systems to communicate with each other over the internet. REST APIs (Application Programming Interfaces) are a way of implementing REST principles in web applications.
- REST APIs are designed to be stateless, meaning that each request sent to the API should contain all the necessary information for the API to understand and process it. The API responds with the requested data in a format that is easily consumed by the client, typically in JSON or XML format.
- REST APIs typically use HTTP methods, such as GET, POST, PUT, and DELETE, to represent different types of operations on the resources exposed by the API. For example, a GET request might be used to retrieve data from the API, while a POST request might be used to create new data.

- REST APIs can be used to build a wide range of web applications, from simple web pages to complex applications with multiple services and integrations. They are often used in combination with other technologies, such as JavaScript frameworks
- like React, Angular, or Vue.js, and backend technologies like Node.js and MongoDB.
- One of the advantages of REST APIs is that they provide a standard interface for different systems to communicate with each other, making it easier to integrate different services and technologies. They also provide a scalable architecture that can handle large amounts of data and requests.
- In summary, REST APIs are a popular way of implementing the REST architectural style in web applications. They provide a standard interface for different systems to communicate with each other and can be used to build a wide range of applications. Their stateless and scalable architecture make them a popular choice for building web services that can handle large amounts of data and requests.

6.2 PROCESS:

User Authentication:

User Authentication and Authorization This module handles signup, login, forgot password, OTP/reset flow, and secure session/token validation .It supports role-based access for Buyer (user), Seller/Agent ,and Admin .It also controls protected pages such as dashboard, post property, inquiries, and admin features.

Property and User Data Management:

This module manages core data entities used in Urban-Stay, including Users, Properties, Projects, Inquiries, Reviews, Payments, and Alerts. It supports create, update, verify, and delete operations with validation and access control. Admin can manage user status, listing approvals, featured properties, and data quality. Seller/Agent can manage property listings, media, pricing, and availability details.

Inquiry, Appointment, and Communication Management:

This module handles buyer-to-seller communication through inquiry threads and appointment requests (schedule-visit flow).

It tracks inquiry status such as pending, responded, scheduled, completed, and cancelled.

It supports message responses, visit date/time preference, and read-status tracking for better follow-up.

Notifications and Updates:

This module sends important updates to users, sellers, and admins such as: new inquiry alerts, appointment status updates, listing approval/rejection, featured plan/payment status, and system announcements. It improves response time and user engagement across the platform.

Payment and Featured Listing Management:

This module records transactions for featured listing plans, tracks payment status (captured/failed/refunded), and manages featured validity dates. It ensures secure and auditable billing history for seller-side monetization.

Reporting and Monitoring (Admin Side):

This module gives admin operational visibility into users, listings, inquiries, bookings, reviews, and revenue indicators .It helps in decision-making, moderation, and performance tracking of the platform.

6.3 FINDING / RESULTS / OUTCOME:

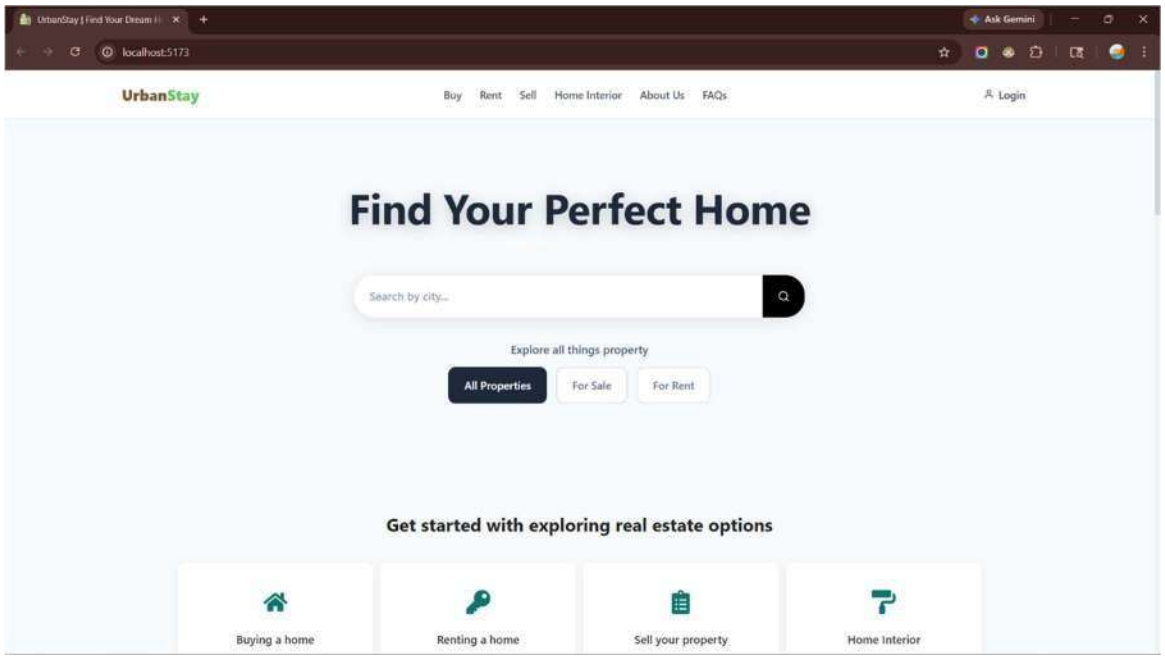

Fig 6.1 Home page 1

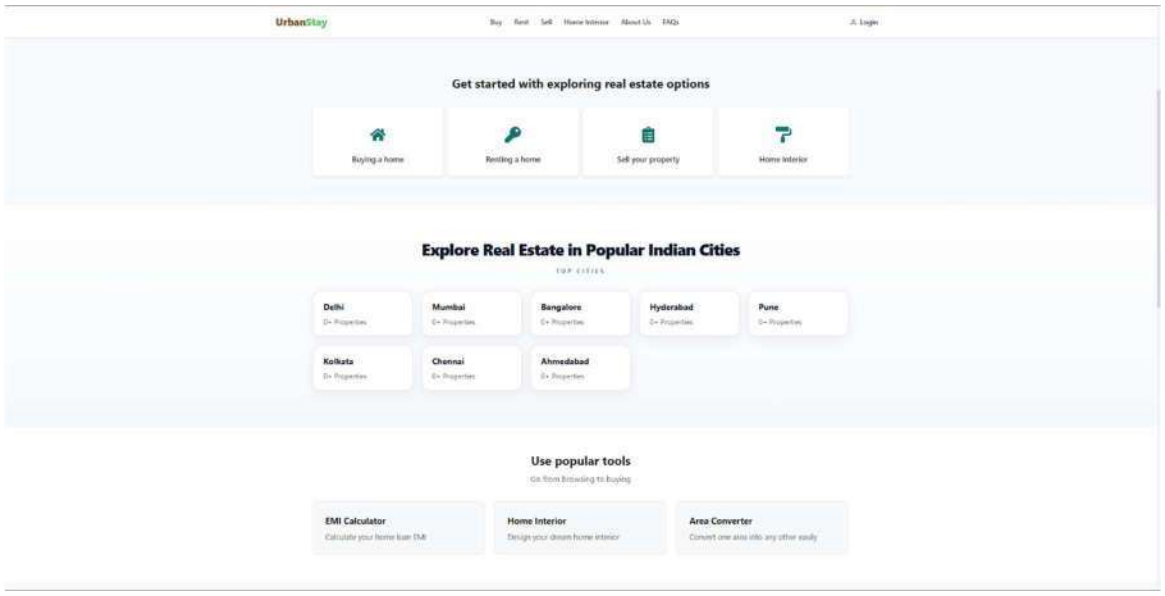

Fig 6.2 Home page 2

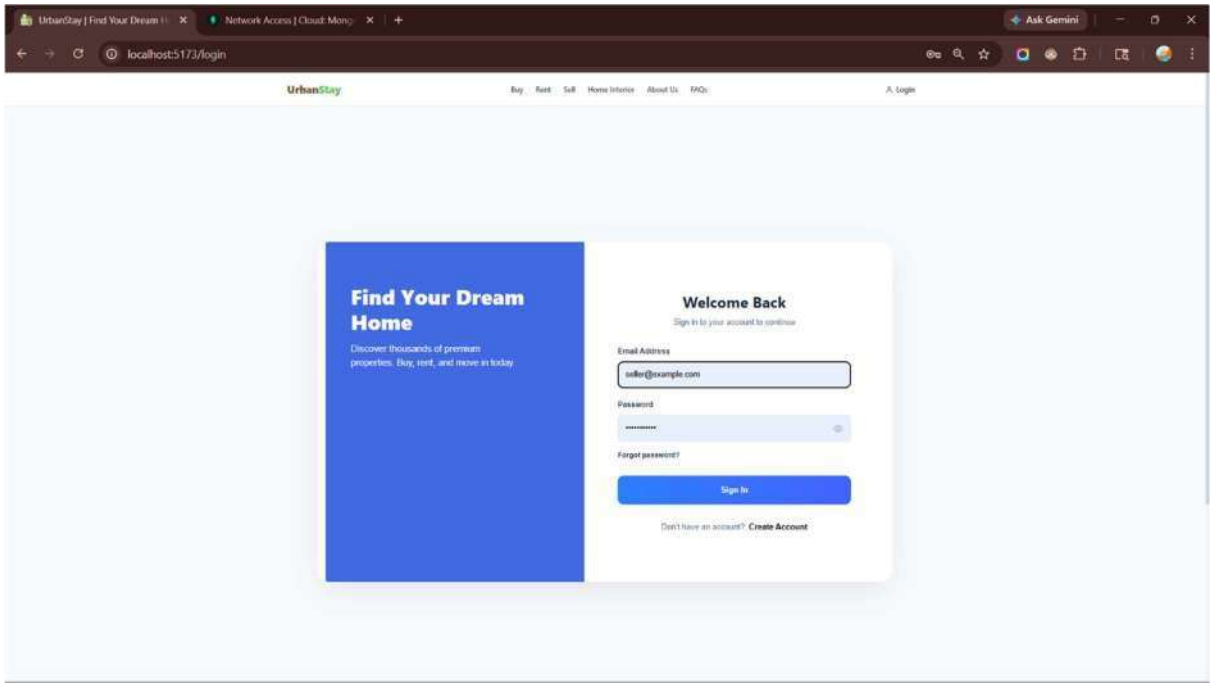

Fig 6.3 Admin Login

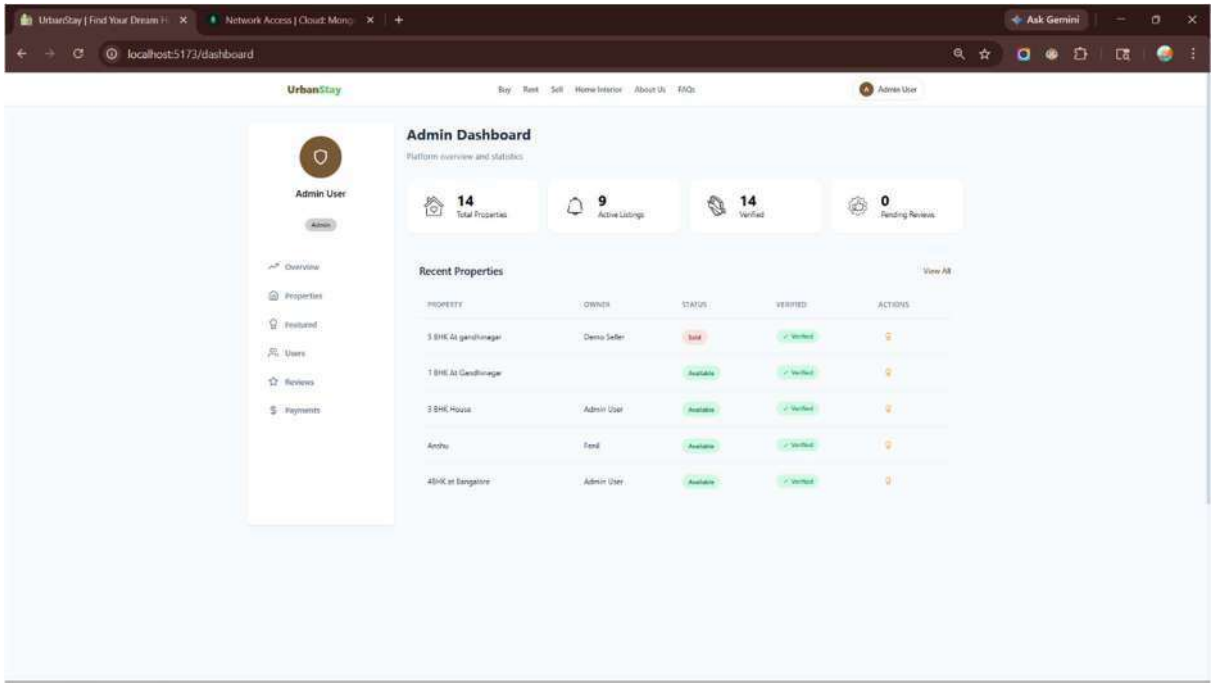

Fig 6.4 Admin Dashboard

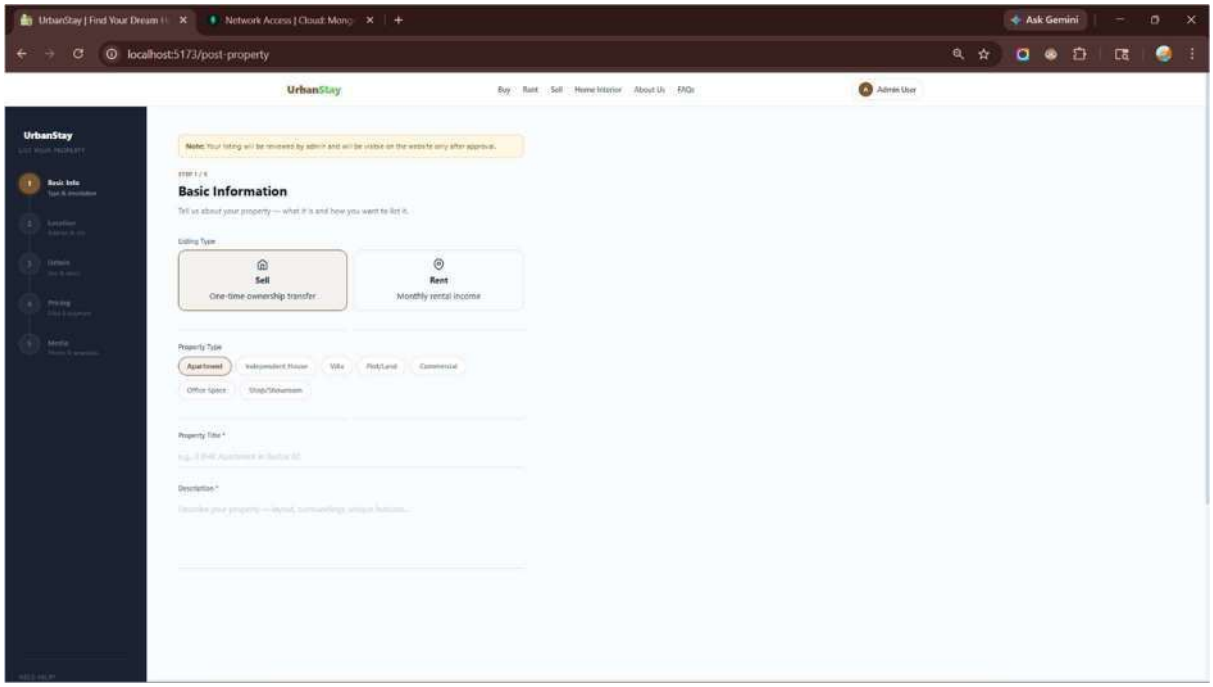

Fig 6.5 For Selling Property

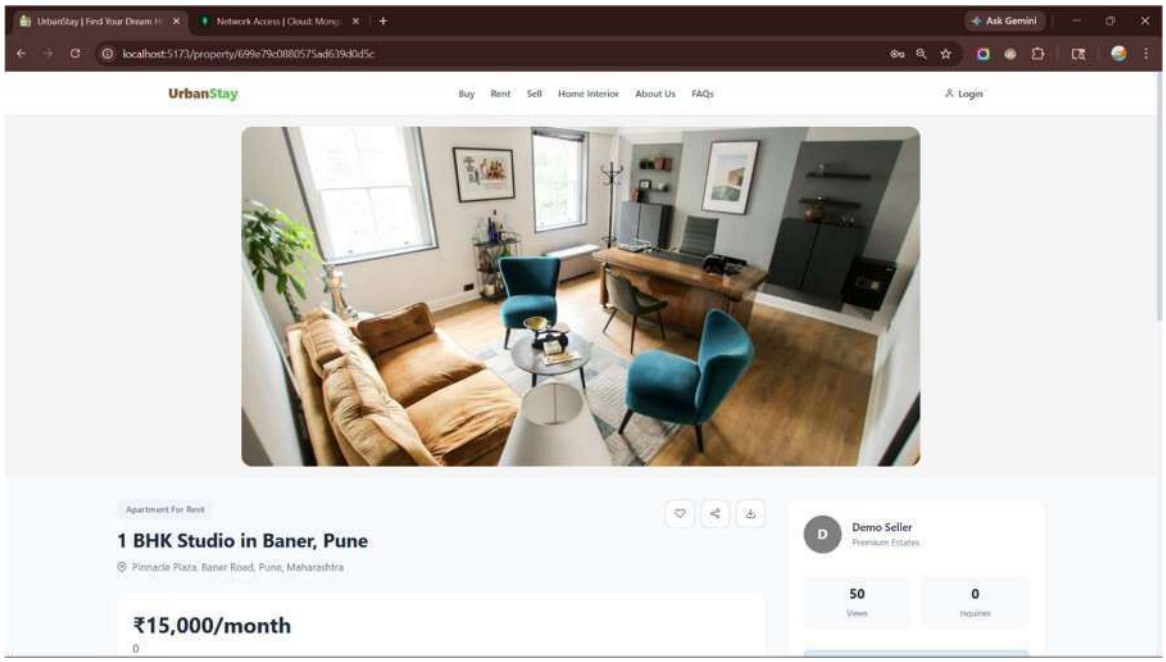

Fig 6.6 Property Details

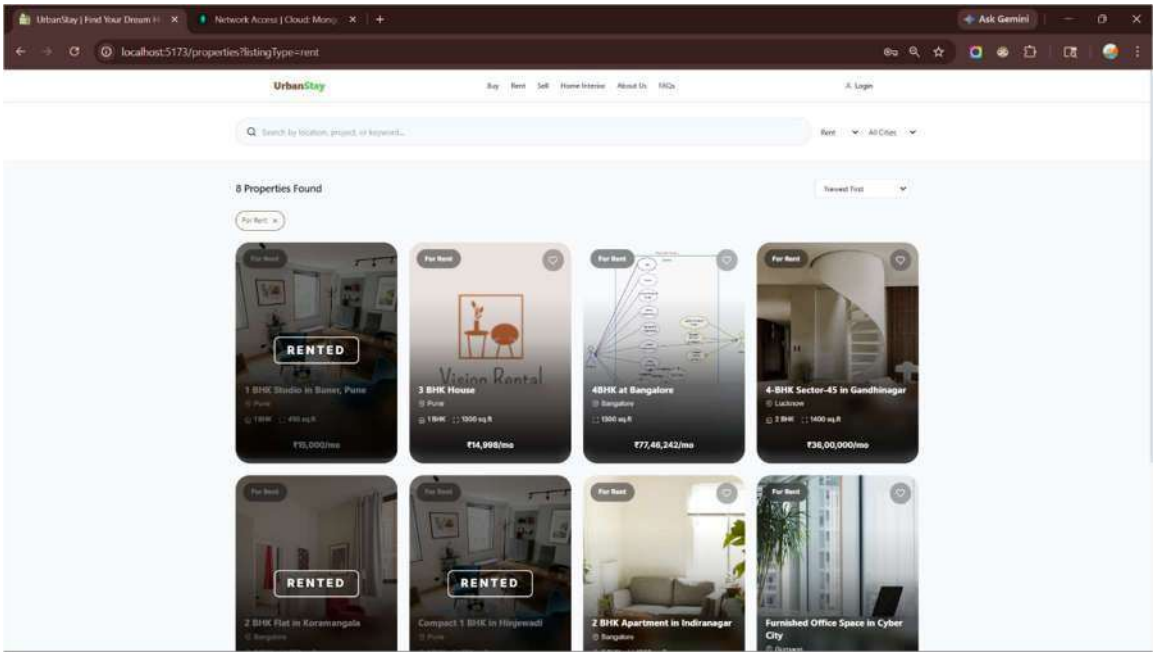

Fig 6.7 Property Card

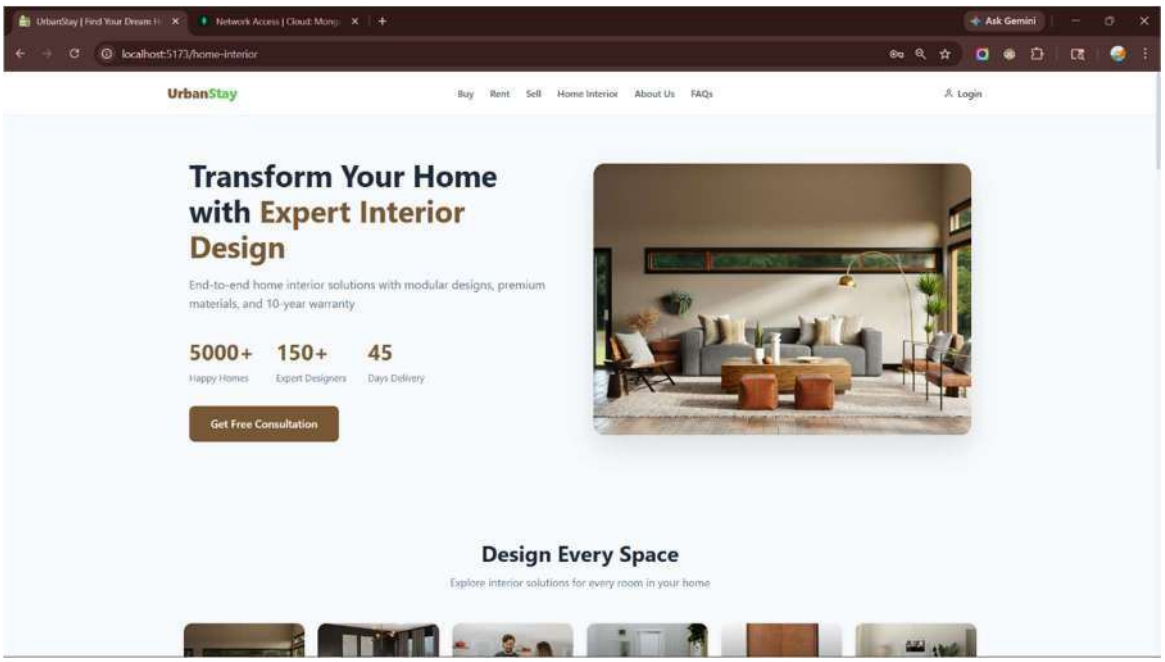

Fig 6.8 Home Interior

CHAPTER 7: TESTING

7.1 TESTING PLAN / STRATEGY

Developing a testing strategy for a MERN (Mongodb, Express.js, React, Node.js) project involves similar steps to a React project but with additional considerations for the backend components. Here's how you might approach it.

- **Identify Components:** Just as with React, identify the key components of your MERN project, including frontend UI components, backend API endpoints, database interactions, and any external services or integrations.
- **Frontend Testing:**

1. **Unit Testing:** Write unit tests for individual React components and utility functions using tools like Jest and React Testing Library.

2. **Integration Testing:** Test how different frontend components interact with each other using tools like Jest and Enzyme. Mock external dependencies and API calls to isolate the frontend for testing.

- **Backend Testing:**

1. **Unit Testing:** Write unit tests for each endpoint of your Express.js API using tools like Mocha, Chai, or Jest. Mock database interactions and external services to test endpoint logic in isolation.

2 Integration Testing: Test how different parts of your backend system interact with each other. This may involve testing database interactions, middleware, and error handling.

- **End-to-End Testing:** Perform end-to-end testing to simulate user interactions from the frontend to the backend and the database. Tools like Cypress or Selenium can be used to automate browser-based tests and validate the entire application flow.
- **Database Testing:** Write tests to ensure that database operations (CRUD operations, data validations, etc.) behave as expected. Use tools like Mongoose for MongoDB**.**
- **Performance Testing:** Test the performance of your MERN application under different loads and conditions. Tools like Apache JMeter or Artillery can be used to simulate heavy traffic and measure server response times and throughput.

- **Security Testing:** Conduct security testing to identify vulnerabilities such as injection attacks, cross-site scripting (XSS), and authentication bypasses. Tools like OWASP ZAP or Burp Suite can be used for this purpose.
- **User Acceptance Testing (UAT):** Involve stakeholders or end users in the testing process to gather feedback on the usability and functionality of the application from a holistic perspective.
- **Continuous Integration and Deployment (CI/CD):** Integrate testing into your CI/CD pipeline to automate the testing process and ensure that new code changes are thoroughly tested before being deployed to production.
- **Documentation:** Document your testing strategy, including the types of tests performed, the tools and frameworks used, and any notable findings or issues encountered during testing.

7.2 TEST RESULTS AND ANALYSIS:

Test Cases:

1. Admin Login:

Test Case ID	TC001
Test Case Summary	This is Admin Login.
Prerequisites	Must be authorized user
Test Data	Username: admin Password: password@123
Expected Result	Login Successfully
Actual Result	Login Successfully
Created By	Harsh Rathod
Date Of Creation	30-03-2026
Executed By	Harsh Rathod
Date Of Execution	01-04-2024
Test Environment	OS Windows 10
Status	Pass

Table 7.1: Admin Login

2. User Forgot password:

Test Case ID	TC002
Test Case Summary	This is User Login.
Prerequisites	Must be authorized user
Test Data	User name: Akshit Mahida
Expected Result	Password sent successfully
Actual Result	Login Successfully
Created By	Akshit Mahida
Date Of Creation	21-03-2026
Executed By	Harsh Rathod
Date Of Execution	22-03-2026
Test Environment	OS Windows 10
Status	Pass

Table 7.2: User Forgot password

3. User Click Button/Menu bar:

Test case ID	Test case name	Test Steps	Test Data	Expected Output	Actual Output	status
TC_WEB_001	Launch Web	Launch Web in chrome	website	Display Home page	Display Home page	Pass
TC_WEB_002	Click on Button	Accept new Inquiry	website	Display Button successfully	Display Button successfully	Pass
	Click on Button	Sign In	website			
	Click on Button	Request an appointment	website			

TC_WEB_003	Click on button	Home	Website	Display Button successfully	Display Button successfully	Pass
TC_WEB_003	Click on Menu bar	About us	Website	Display successfully	Display successfully	Pass
		Home Interior	Website			
		FAQs	Website			
TC_WEB_003	Click on button	Search Bar	Website	Display Button successfully	Display Button successfully	Pass
TC_WEB_003	Click on Menu bar	Amenties	Website	Display successfully	Display successfully	Pass
		Reviews	Website			
		contact	Website			

Table 7.3: User Click Button

CHAPTER8:

CONCLUSION AND DISCUSSION

8.1 OVERALL ANALYSIS OF INTERNSHIP:

During the internship first of all they gave the basic knowledge of our languages and then they gave the project. In project first of all we have to design the webpages according they have given as per the SRS (Software Requirements Specification) then we have to design the databases for our website. After designing the database, we have to integrate all the web pages with database and lastly, we have to do testing of our website. Along with the project implementation we were working frontend and backend part simultaneously.

8.2 PROBLEM ENCOUNTER AND POSSIBLE SOLUTION:

The Urban-stay website faces responsiveness issues, including layout breakage on smaller screens, navigation menu misalignment, improperly scaling images, and text readability problems.

Fixed-width elements and unoptimized forms/buttons further hinder mobile usability. To resolve this, implementing CSS media queries, a responsive grid system (Flexbox/Grid), and ensuring images scale with max-width: 100% will enhance adaptability.

Additionally, using relative units (%, px, rem) instead of fixed pixels, optimizing buttons/forms for touchscreens, and testing across multiple devices will ensure a smooth, user-friendly experience on all screen sizes.

8.3 SUMMARY OF INTERNSHIP WORK:

During my internship at Logic Waves Tech, which lasted 3 months, I served as a Full Stack intern. My responsibilities included coding, testing, and debugging software applications, as well as providing support to senior developers in various tasks., where I contributed to projects such Urban Stay.

Throughout my internship, I developed my technical skills in programming languages such as React JS and Tailwind, as well as in software development tools such as Git.

I encountered challenges such as some errors and some systems and query problems, but through perseverance and collaboration with my team, I was able to overcome them.

The internship provided me with valuable insights into Enlighten Infosystems, and I learned MERN technology. I am grateful for the opportunity to have been part of Enlighten

Infosystems and am confident that the experiences gained will serve me well in my future career endeavours.

8.4 LIMITATION AND FUTURE ENHANCEMENTS:

Limitation:

- **Responsiveness Issues:** Some elements may not adapt perfectly across all screen sizes.
- **Limited Features:** Certain functionalities may be basic or missing advanced capabilities
- **Browser Compatibility:** Some design elements might not render correctly in all browsers.

Future Enhancements:

- **Improved Mobile Responsiveness:** Optimize layouts and images for seamless mobile adaptation.
- **Feature Expansion:** Add advanced functionalities such as user authentication, real-time chat, or appointment booking.

CHAPTER 9: REFERNCES :

1. React Documentation. https://react.dev/
2. Vite Documentation. https://vitejs.dev/guide/
3. Node.js Documentation. https://nodejs.org/docs/latest/api/
4. Express.js Documentation. https://expressjs.com/
5. MongoDB Manual. https://www.mongodb.com/docs/manual/
6. Mongoose Documentation. https://mongoosejs.com/docs/
7. ES Lint Documentation. https://eslint.org/docs/latest/
8. Postman Learning Centre. https://learning.postman.com/docs/
9. Cloud nary Documentation (media upload/storage). https://cloudinary.com/documentation